Introduction to Electronic Design Automation

Homework 3

Student: 吳亞倫
ID: B13901011
Department: 電機工程學系

1 BDD Operation

1. (a)

$$
\begin{aligned}
&\text{robdd\_build}(ab(\neg c+d)+(a\neg b+\neg ab)(c\neg d+\neg cd),1)\\
&\xrightarrow{\eta} \text{robdd\_build}(b(\neg c+d)+\neg b(c\neg d+\neg cd),2)\\
&\quad\xrightarrow{\eta} \text{robdd\_build}(\neg c+d,3)\\
&\quad\quad\xrightarrow{\eta} \text{robdd\_build}(d,4)\\
&\quad\quad\quad\xrightarrow{\eta} \text{robdd\_build}(1,5)\\
&\quad\quad\quad\quad v_1\\
&\quad\quad\quad\xrightarrow{\lambda} \text{robdd\_build}(0,5)\\
&\quad\quad\quad\quad v_0\\
&\quad\quad\quad v_2=(d,v_1,v_0)\\
&\quad\quad\xrightarrow{\lambda} \text{robdd\_build}(1,4)\\
&\quad\quad\quad v_1\\
&\quad\quad v_3=(c,v_2,v_1)\\
&\quad\xrightarrow{\lambda} \text{robdd\_build}(c\neg d+\neg cd,3)\\
&\quad\quad\xrightarrow{\eta} \text{robdd\_build}(\neg d,4)\\
&\quad\quad\quad\xrightarrow{\eta} \text{robdd\_build}(0,5)\\
&\quad\quad\quad\quad v_0\\
&\quad\quad\quad\xrightarrow{\lambda} \text{robdd\_build}(1,5)\\
&\quad\quad\quad\quad v_1\\
&\quad\quad\quad v_5=(d,v_0,v_1)\\
&\quad\quad\xrightarrow{\lambda} \text{robdd\_build}(d,4)\\
&\quad\quad\quad v_2=(d,v_1,v_0)\\
&\quad\quad v_4=(c,v_5,v_2)\\
&\quad v_6=(b,v_3,v_4)\\
&\xrightarrow{\lambda} \text{robdd\_build}(b(c\neg d+\neg cd),2)\\
&\quad\xrightarrow{\eta} \text{robdd\_build}(c\neg d+\neg cd,3)\\
&\quad\quad v_4=(c,v_5,v_2)\\
&\quad\xrightarrow{\lambda} \text{robdd\_build}(0,3)\\
&\quad\quad v_0\\
&\quad v_7=(b,v_4,v_0)\\
&v_8=(a,v_6,v_7)
\end{aligned}
$$

1. (b)

Since

$$f = ab(\neg c + d) + (a\neg b + \neg ab)(c\neg d + \neg cd)$$

we have

$$f_c = abd + (a\neg b + \neg ab)(\neg d)$$
$$f_{\neg c} = ab + (a\neg b + \neg ab)(d)$$

Using procedure `robdd_build` to f_c, we have

$$\begin{aligned}
&\text{robdd_build}(abd + (a\neg b + \neg ab)(\neg d), 1)\\
&\xrightarrow{\eta} \text{robdd_build}(bd + \neg b\neg d, 2)\\
&\quad\xrightarrow{\eta} \text{robdd_build}(d, 4)\\
&\quad\quad v_2 = (d, v_1, v_0)\\
&\quad\xrightarrow{\lambda} \text{robdd_build}(\neg d, 4)\\
&\quad\quad v_5 = (d, v_0, v_1)\\
&\quad v_9 = (b, v_2, v_5)\\
&\xrightarrow{\eta} \text{robdd_build}(b\neg d, 2)\\
&\quad\xrightarrow{\eta} \text{robdd_build}(\neg d, 4)\\
&\quad\quad v_5 = (d, v_0, v_1)\\
&\quad\xrightarrow{\lambda} \text{robdd_build}(0, 4)\\
&\quad\quad v_0\\
&\quad v_{10} = (b, v_5, v_0)\\
&v_{11} = (a, v_9, v_{10})
\end{aligned}$$

Using procedure `robdd_build` to $f_{\neg c}$, we have

$$\begin{aligned}
&\text{robdd_build}(ab + (a\neg b + \neg ab)d, 1)\\
&\xrightarrow{\eta} \text{robdd_build}(b + \neg bd, 2)\\
&\quad\xrightarrow{\eta} \text{robdd_build}(1, 4)\\
&\quad\quad v_1\\
&\quad\xrightarrow{\lambda} \text{robdd_build}(d, 4)\\
&\quad\quad v_2 = (d, v_1, v_0)\\
&\quad v_{12} = (b, v_1, v_2)\\
&\xrightarrow{\lambda} \text{robdd_build}(bd, 2)\\
&\quad\xrightarrow{\eta} \text{robdd_build}(d, 4)\\
&\quad\quad v_2 = (d, v_1, v_0)\\
&\quad\xrightarrow{\lambda} \text{robdd_build}(0, 4)\\
&\quad\quad v_0\\
&\quad v_{13} = (b, v_2, v_0)\\
&v_{14} = (a, v_{12}, v_{13})
\end{aligned}$$

Hence, the shared ROBDDs of f, f_c and $f_{\neg c}$ are as shown as below (Figure 1).

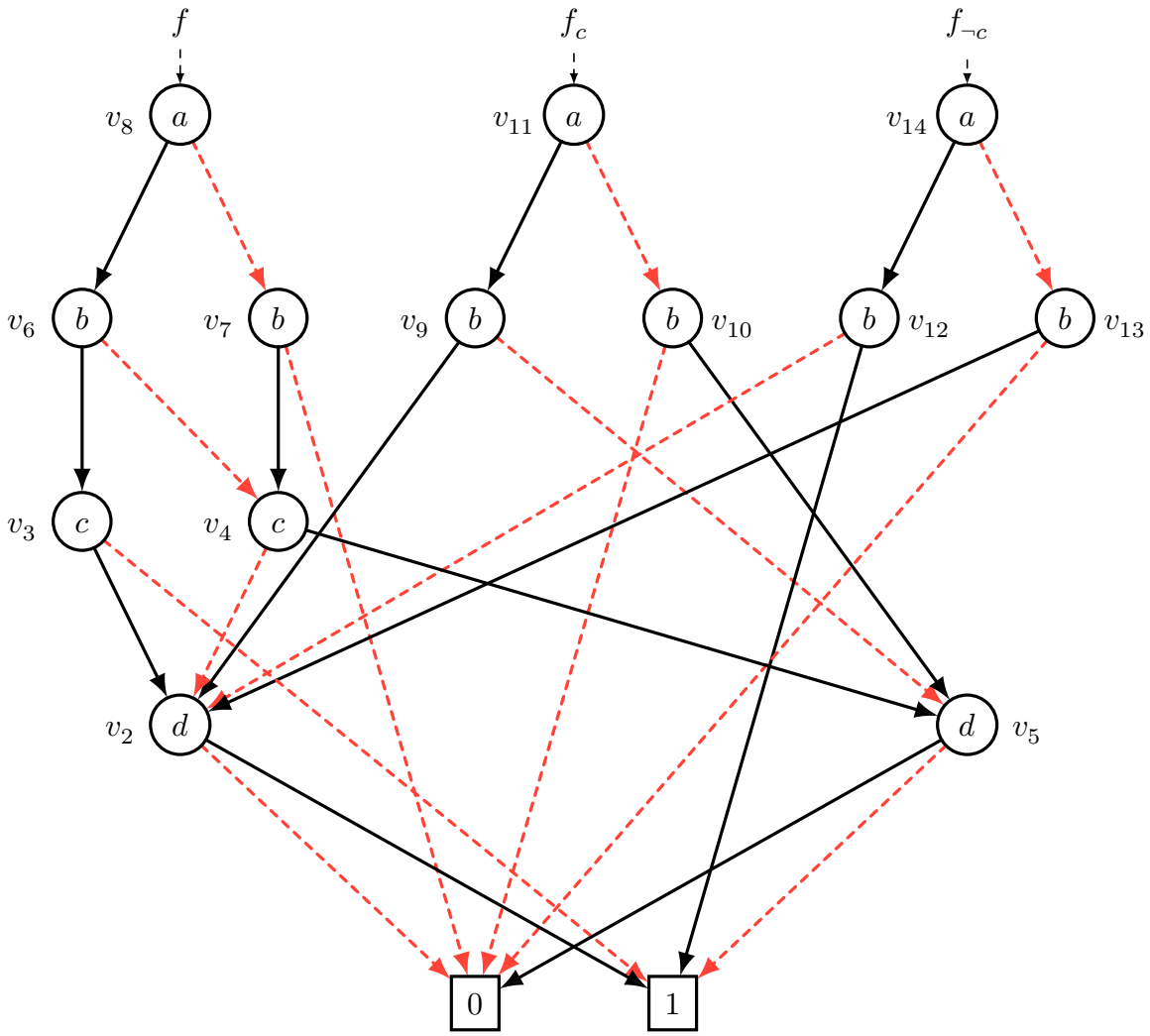


Figure 1: The shared ROBDDs of f, f_c and $f_{\neg c}$.
Red dashed edge represents 0-edge, and black solid edge represents 1-edge.

1. (c)

Since

$$\exists c.f = f_c + f_{\neg c} = \text{ITE}(f_c, 1, f_{\neg c}) = \text{ITE}(v_{11}, 1, v_{14})$$

we can use the ROBDD in Figure 1 to compute the ROBDD of $\text{ITE}(f_c, 1, f_{\neg c})$. Hence we have:

$$\begin{aligned}
&\text{ITE}(f_c, 1, f_{\neg c}) \\
&= \text{ITE}(v_{11}, 1, v_{14}) \\
&= \text{ITE}(a,\ \ \text{ITE}(v_9, 1, v_{12}),\ \ \text{ITE}(v_{10}, 1, v_{13}))) \\
&= \text{ITE}(a,\ \ \text{ITE}(b,\ \ \text{ITE}(v_2, 1, 1),\ \ \text{ITE}(v_5, 1, v_2)),\ \ \text{ITE}(b,\ \ \text{ITE}(v_5, 1, v_2),\ \ \text{ITE}(0, 1, 0)))
\end{aligned}$$

since $v_2 = d$ and $v_5 = \neg d$, we have

$$\begin{aligned}
&\text{ITE}(v_2, 1, 1) = 1 \\
&\text{ITE}(v_5, 1, v_2) = \text{ITE}(\neg d, 1, d) = 1 \\
&\text{ITE}(0, 1, 0) = 0.
\end{aligned}$$

Therefore,

$$
\begin{aligned}
&\text{ITE}(f_c, 1, f_{\neg c}) \\
&= \text{ITE}(a,\ \ \text{ITE}(b, 1, 1),\ \ \text{ITE}(b, 1, 0)) \\
&= \text{ITE}(a, 1, b) \\
&= a + b
\end{aligned}
$$

The ROBDD of $\text{ITE}(f_c, 1, f_{\neg c}) = \text{ITE}(a, 1, b) = a + b$ is as shown as below (Figure 2).

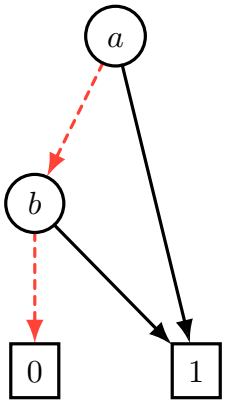


Figure 2: The ROBDD of $\text{ITE}(f_c, 1, f_{\neg c})$.

2 SAT Solving

$C_1 = (a + b + c)$, $C_2 = (a + \neg c + \neg d + e)$, $C_3 = (\neg b + c + \neg d + e)$,
$C_4 = (a + \neg c + \neg e)$, $C_5 = (\neg c + d + e)$, $C_6 = (\neg b + c + d)$,
$C_7 = (a + \neg b + c + \neg d + \neg e)$, $C_8 = (\neg a + b + c + e)$, $C_9 = (\neg a + \neg c + d + \neg e)$.

2. (a)

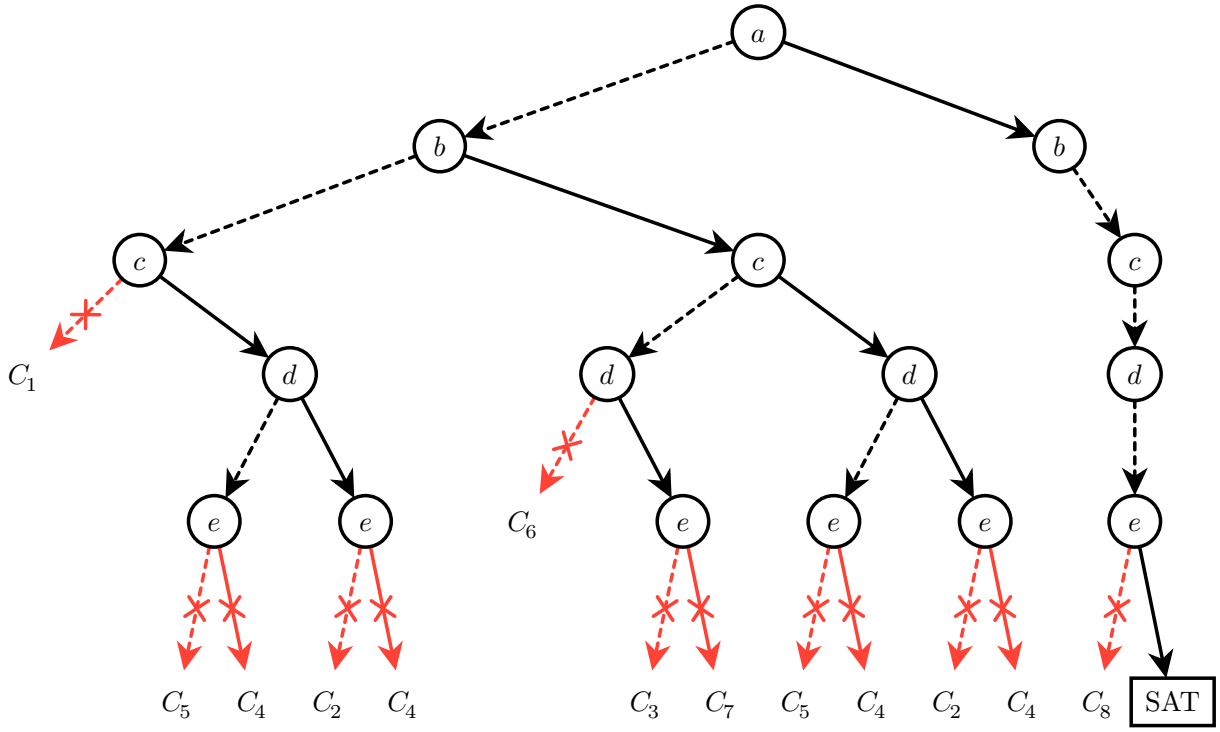


Figure 3: The search tree in solving the above CNF formula without implication and conflict-based learning.

2. (b)

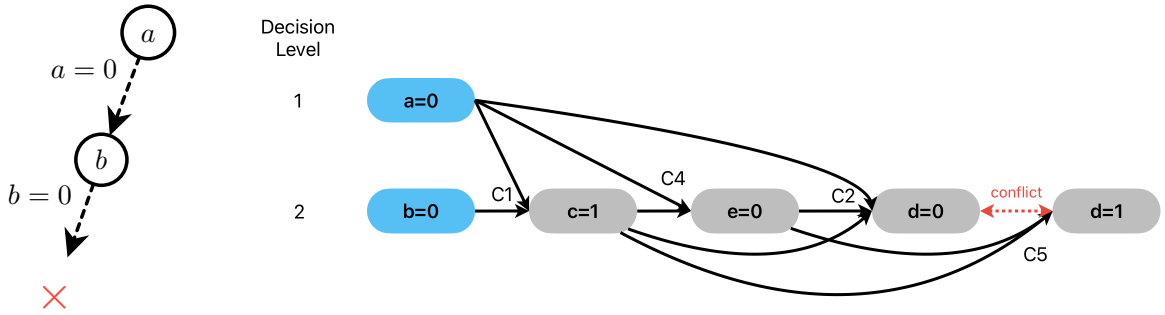
Graphical tree are hard to show both the search process and implication, hence we use the following text-based tree to show the search process and implication in solving the above CNF formula without implication and conflict-based learning. In the tree, each node represents a variable assignment, and each edge represents an implication. The leaf nodes with "✓ SAT" represent a satisfying assignment, and the leaf nodes with "✗ conflict at C_x" represent a conflict at clause C_x.

```
root
├── a=0
│   ├── b=0  => c=1(C1), e=0(C4), d=0(C2)  ✗ conflict at C5
│   └── b=1
│       ├── c=0  => d=1(C6), e=1(C3)  ✗ conflict at C7
│       └── c=1  => e=0(C4), d=0(C2)  ✗ conflict at C5
└── a=1
    └── b=0
        └── c=0  => e=1(C8)
            ├── d=0  ✓ SAT
            └── d=1  ✓ SAT
```

2. (c)

The search tree in solving the above CNF formula with implication and conflict-based learning is shown as below.

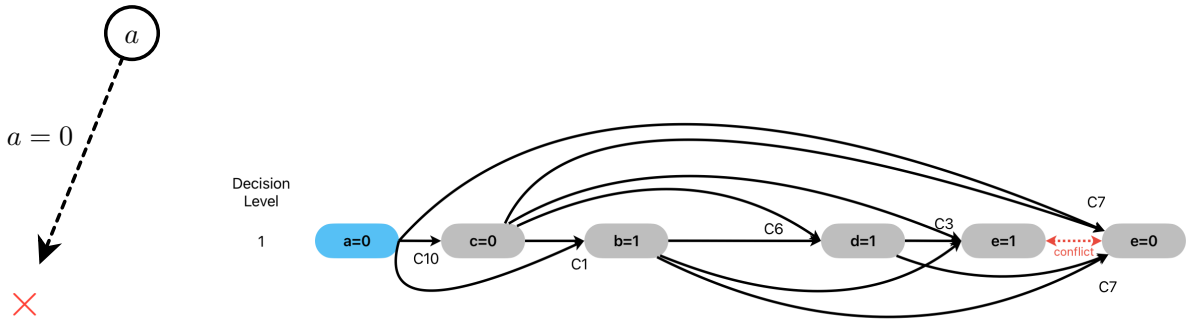
- **First conflict**:



(a) Search Tree (b) Implication graph

Figure 4: The search tree and implication graph of first conflict in solving

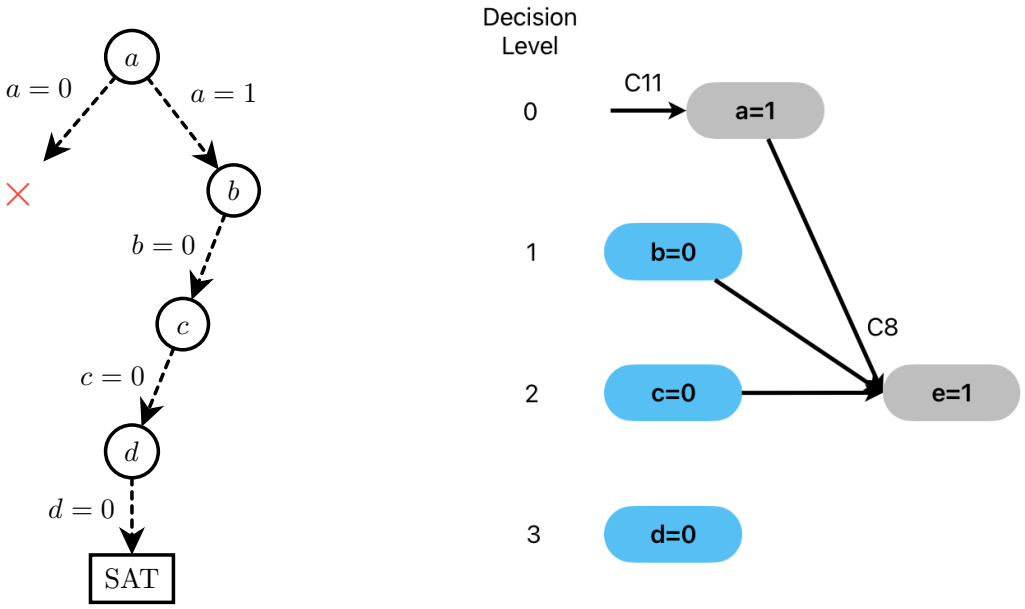
  - Learning clauses:
    - Resolve C_2 and C_5 on d, we learned $R_1 = (a + \neg c + e)$.
    - Resolve C_4 and R_1 on e, we learned $R_2 = (a + \neg c)$.
    - Resolve C_1 and R_2 on c, we learned $R_3 = (a + b)$.
  - The possible learned clause candidates are R_1, R_2, and R_3. We choose R_2 because it contains only one literal at the current decision level.
  - We add $C_{10} = R_2 = (a + \neg c)$ to the clause set, and backtrack to the decision level of 1.
- **Second conflict**:



(a) Search Tree (b) Implication graph

Figure 5: The search tree and implication graph of second conflict in solving

  - Learning clauses:
    - Resolve C_3 and C_7 on e, we learned $R_4 = (a + \neg b + c + \neg d)$.
    - Resolve C_6 and R_4 on d, we learned $R_5 = (a + \neg b + c)$.
    - Resolve C_1 and R_5 on b, we learned $R_6 = (a + c)$.
    - Resolve C_{10} and R_6 on c, we learned $R_7 = a$.
  - The possible learned clause candidates are R_4, R_5, R_6, and R_7. We choose R_7 because it contains only one literal at the current decision level.
  - We add $C_{11} = R_7 = a$ to the clause set, and backtrack to decision level 0. Since $C_{11} = a$ is a unit clause, it implies $a = 1$.
- **SAT**:



(a) Search Tree (b) Implication graph

Figure 6: The search tree and implication graph of SAT assignment in solving

- Therefore, we find a satisfying assignment $(a, b, c, d, e) = (1, 0, 0, 0, 1)$. Hence, the CNF formula is SAT.

3 Combinational Equivalence Checking

3. (a)

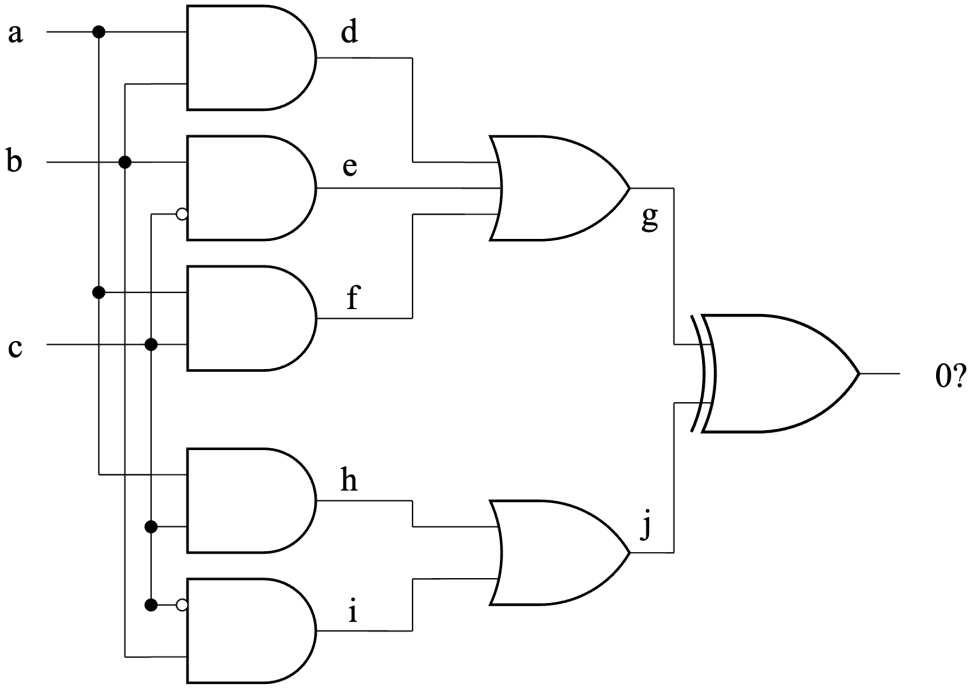


Figure 7: The corresponding miter circuit for equivalence checking.

3. (b)

Since we have 3 primary inputs, the total possible input combinations are $2^3 = 8$. So we only need one iteration of 8-bit parallelsimulation to achieve exhaustive simulation on the miter circuit of Figure 7.

$$a = 10101010 \quad (0xAA);$$
$$b = 11001100 \quad (0xCC);$$
$$c = 11110000 \quad (0xF0);$$

$$d = a \wedge b = 10101010 \;\&\; 11001100 = 10001000 \quad (0x88);$$
$$e = b \wedge \neg c = 11001100 \;\&\; 00001111 = 00001100 \quad (0x0C);$$
$$f = c \wedge a = 11110000 \;\&\; 10101010 = 10100000 \quad (0xA0);$$
$$g = d \vee e \vee f = 10001000 \mid 00001100 \mid 10100000 = 10101100 \quad (0xAC);$$

$$h = a \wedge c = 11110000 \;\&\; 10101010 = 10100000 \quad (0xA0);$$
$$i = b \wedge \neg c = 11001100 \;\&\; 00001111 = 00001100 \quad (0x0C);$$
$$j = h \vee j = 10100000 \mid 00001100 = 10101100 \quad (0xAC);$$

$$\text{Final Output} = g \oplus j = 10101100 \oplus 10101100 = 00000000 \quad (0x00).$$

Since the final output is always 0, we can conclude that the two circuits are combinationally equivalent.

3. (c)

The miter output is z. To formulate the equivalence checking problem as a SAT instance, we force the miter output to be 1. Therefore, the SAT instance is the CNF formula Phi and z, where Phi encodes all gates in the miter circuit.

From the circuit, the gate relations are

$$d = a \wedge b$$
$$e = b \wedge \neg c$$
$$f = a \wedge c$$
$$g = d \vee e \vee f$$
$$h = a \wedge c$$
$$i = b \wedge \neg c$$
$$j = h \vee i$$
$$z = g \oplus j$$

Each gate is translated into CNF as follows.

$$d = a \wedge b \quad \Rightarrow \quad (\neg d \vee a) \wedge (\neg d \vee b) \wedge (\neg a \vee \neg b \vee d)$$
$$e = b \wedge \neg c \quad \Rightarrow \quad (\neg e \vee b) \wedge (\neg e \vee \neg c) \wedge (\neg b \vee c \vee e)$$
$$f = a \wedge c \quad \Rightarrow \quad (\neg f \vee a) \wedge (\neg f \vee c) \wedge (\neg a \vee \neg c \vee f)$$
$$g = d \vee e \vee f \quad \Rightarrow \quad (\neg d \vee g) \wedge (\neg e \vee g) \wedge (\neg f \vee g) \wedge (\neg g \vee d \vee e \vee f)$$
$$h = a \wedge c \quad \Rightarrow \quad (\neg h \vee a) \wedge (\neg h \vee c) \wedge (\neg a \vee \neg c \vee h)$$
$$i = b \wedge \neg c \quad \Rightarrow \quad (\neg i \vee b) \wedge (\neg i \vee \neg c) \wedge (\neg b \vee c \vee i)$$
$$j = h \vee i \quad \Rightarrow \quad (\neg h \vee j) \wedge (\neg i \vee j) \wedge (\neg j \vee h \vee i)$$
$$z = g \oplus j \quad \Rightarrow \quad (g \vee j \vee \neg z) \wedge (g \vee \neg j \vee z) \wedge (\neg g \vee j \vee z) \wedge (\neg g \vee \neg j \vee \neg z)$$

Therefore, the final SAT instance is

$$\begin{aligned}\Phi = {} & (\neg d \vee a) \wedge (\neg d \vee b) \wedge (\neg a \vee \neg b \vee d) \\ & \wedge (\neg e \vee b) \wedge (\neg e \vee \neg c) \wedge (\neg b \vee c \vee e) \\ & \wedge (\neg f \vee a) \wedge (\neg f \vee c) \wedge (\neg a \vee \neg c \vee f) \\ & \wedge (\neg d \vee g) \wedge (\neg e \vee g) \wedge (\neg f \vee g) \wedge (\neg g \vee d \vee e \vee f) \\ & \wedge (\neg h \vee a) \wedge (\neg h \vee c) \wedge (\neg a \vee \neg c \vee h) \\ & \wedge (\neg i \vee b) \wedge (\neg i \vee \neg c) \wedge (\neg b \vee c \vee i) \\ & \wedge (\neg h \vee j) \wedge (\neg i \vee j) \wedge (\neg j \vee h \vee i) \\ & \wedge (g \vee j \vee \neg z) \wedge (g \vee \neg j \vee z) \wedge (\neg g \vee j \vee z) \wedge (\neg g \vee \neg j \vee \neg z) \\ & \wedge z\end{aligned}$$

The last clause z forces the miter output to be 1. Thus, the SAT solver searches for an assignment such that the two circuit outputs are different. If this CNF formula is satisfiable, then the satisfying assignment is a counterexample to equivalence. If this CNF formula is unsatisfiable, then no such counterexample exists, and the two circuits are equivalent.

3. (d)

I translate the above CNF formula into DIMACS format as follows. note that we use the following variable mapping: $a = 1, b = 2, c = 3, d = 4, e = 5, f = 6, g = 7, h = 8, i = 9, j = 10, z = 11$.

```
DIMACS format for the miter equivalence checking problem
p cnf 11 27
-4 1 0
-4 2 0
-1 -2 4 0
-5 2 0
-5 -3 0
-2 3 5 0
-6 1 0
-6 3 0
-1 -3 6 0
-4 7 0
-5 7 0
-6 7 0
-7 4 5 6 0
-8 1 0
-8 3 0
-1 -3 8 0
-9 2 0
-9 -3 0
-2 3 9 0
-8 10 0
-9 10 0
-10 8 9 0
7 10 -11 0
7 -10 11 0
-7 10 11 0
-7 -10 -11 0
11 0
```

The result of running `minisat` on the above CNF file is `UNSAT`, which means that the two circuits are combinationally equivalent.

```
❯ minisat miter_equivalence.cnf
============================[ Problem Statistics ]=============================
|                                                                             |
|  Number of variables:            11                                         |
|  Number of clauses:              26                                         |
|  Parse time:                  0.00 s                                        |
|  Eliminated clauses:          0.00 Mb                                       |
|  Simplification time:         0.00 s                                        |
|                                                                             |
===============================================================================
Solved by simplification
restarts              : 0
conflicts             : 0             (0 /sec)
decisions             : 0             ( nan % random) (0 /sec)
propagations          : 2             (596 /sec)
conflict literals     : 0             ( nan % deleted)
CPU time              : 0.003354 s

UNSATISFIABLE
```

Figure 8: The result of running `minisat` on the above CNF file.

3. (e)

I write the two circuits in Figure 1 into BLIF format as follows.

The BLIF file of the first circuit is:

```
# C1: g = (a & b) | (b & ~c) | (a & c)
.model c1
.inputs a b c
.outputs z

# d = a & b
.names a b d
11 1

# e = b & ~c
.names b c e
10 1

# f = a & c
.names a c f
11 1

# z = d | e | f
.names d e f z
1-- 1
-1- 1
--1 1

.end
```

The BLIF file of the second circuit is:

```
# C2: j = (a & c) | (b & ~c)
.model c2
.inputs a b c
.outputs z

# h = a & c
.names a c h
11 1

# i = b & ~c
.names b c i
10 1

# z = h | i
.names h i z
1- 1
-1 1

.end
```

Then I run ABC with `abc -c "cec c1.blif c2.blif"`, and the result is `c1.blif and c2.blif are equivalent.` Therefore, the two circuits are combinationally equivalent.

```
❯ abc -c "cec c1.blif c2.blif"
======== ABC command line "cec c1.blif c2.blif"
Networks are equivalent.  Time =     0.01 sec
```

Figure 9: The result of running ABC for combinational equivalence checking.

4 Characteristic Function

4. (a)

The set of states each of which has no direct transition to itself has characteristic function of

$$F(\boldsymbol{s}) = \neg T(\boldsymbol{s}, \boldsymbol{s})$$

4. (b)

The set of states that can be reached from C in exactly two transitions has characteristic function of

$$G(\boldsymbol{s}) = \exists \boldsymbol{s_1}, \boldsymbol{s_2}.C(\boldsymbol{s_1}) \wedge T(\boldsymbol{s_1}, \boldsymbol{s_2}) \wedge T(\boldsymbol{s_2}, \boldsymbol{s})$$

4. (c)

The set of states that each have a unique next state has characteristic function of

$$H(\boldsymbol{s}) = \exists \boldsymbol{s_1}.(T(\boldsymbol{s}, \boldsymbol{s_1}) \wedge \forall \boldsymbol{s_2}.((T(\boldsymbol{s}, \boldsymbol{s_1}) \wedge T(\boldsymbol{s}, \boldsymbol{s_2})) \rightarrow \boldsymbol{s_1} = \boldsymbol{s_2}))$$

5 Sequential Equivalence Checking

5. (a)

Transition function of C_1:

$$s_{1'} = \neg((\neg(x_1 \wedge x_2)) \wedge (\neg s_1)) = (x_1 \wedge x_2) \vee s_1$$

Output function of C_1:

$$z_1 = s_1$$

Transition functions of C_2:

$$s_{2'} = x_1$$
$$s_{3'} = x_2$$
$$s_{4'} = (s_2 \wedge s_3) \vee s_4$$

Output function of C_2:

$$z_2 = (s_2 \wedge s_3) \vee s_4$$

Since a characteristic function is 1 exactly on the corresponding initial state, and the initial values are $r_1 = 0, r_2 = 1, r_3 = 0, r_4 = 0$, we have:

Characteristic function of the initial state of C_1:

$$I_1(s_1) = \neg s_1$$

Characteristic function of the initial state of C_2:

$$I_2(s_2, s_3, s_4) = s_2 \wedge \neg s_3 \wedge \neg s_4$$

5. (b)

Transition functions of product machine $C_{1\times2}$:

$$s_1' = (x_1 \wedge x_2) \vee s_1$$

$$s_2' = x_1$$

$$s_3' = x_2$$

$$s_4' = (s_2 \wedge s_3) \vee s_4$$

Output functions of product machine $C_{1\times2}$:

$$z_1 = s_1$$

$$z_2 = (s_2 \wedge s_3) \vee s_4$$

$$z = z_1 \oplus z_2 = s_1 \oplus ((s_2 \wedge s_3) \vee s_4)$$

Since the initial values are $r_1 = 0, r_2 = 1, r_3 = 0, r_4 = 0$, the initial state of $C_{1\times2}$ is

$$(s_1, s_2, s_3, s_4) = (0, 1, 0, 0)$$

Its characteristic function is

$$I_{1\times2}(s_1, s_2, s_3, s_4) = \neg s_1 \wedge s_2 \wedge \neg s_3 \wedge \neg s_4$$

5. (c)

Transition relation of $C_{1\times2}$ is

$$\begin{aligned} T(\boldsymbol{x}, \boldsymbol{s}, \boldsymbol{s}') &= \prod_i \left(s_{i'} \equiv \delta_{i(\vec{x},\vec{s})}\right) \\ &= (s_1' \equiv (s_1 \vee (x_1 \wedge x_2))) \wedge (s_2' \equiv x_1) \wedge (s_3' \equiv x_2) \wedge (s_4' \equiv ((s_2 \wedge s_3) \vee s_4)), \end{aligned}$$

Quantified Transition Relation:

$$\begin{aligned} T_\exists(\boldsymbol{s}, \boldsymbol{s}') &= \exists \boldsymbol{x}.T(\boldsymbol{x}, \boldsymbol{s}, \boldsymbol{s}') \\ &= \exists x_1 \exists x_2.(s_1' \equiv (s_1 \vee (x_1 \wedge x_2))) \wedge (s_2' \equiv x_1) \wedge (s_3' \equiv x_2) \wedge (s_4' \equiv ((s_2 \wedge s_3) \vee s_4)) \end{aligned}$$

Since

$$\exists x.((x \equiv a) \wedge F(x)) \equiv F(a),$$

we can eliminate x_1 and x_2 as follows:

$$T_\exists(\boldsymbol{s}, \boldsymbol{s}') = \exists x_1 \exists x_2.((s_1' \equiv (s_1 \vee (x_1 \wedge x_2))) \wedge (s_2' \equiv x_1) \wedge (s_3' \equiv x_2) \wedge (s_4' \equiv ((s_2 \wedge s_3) \vee s_4)))$$

First eliminate x_1:

$$= \exists x_2.((s_1' \equiv (s_1 \vee (s_2' \wedge x_2))) \wedge (s_3' \equiv x_2) \wedge (s_4' \equiv ((s_2 \wedge s_3) \vee s_4)))$$

Then eliminate x_2:

$$= (s_1' \equiv (s_1 \vee (s_2' \wedge s_3'))) \wedge (s_4' \equiv ((s_2 \wedge s_3) \vee s_4))$$

Hence, the quantified transition relation is

$$T_\exists(\boldsymbol{s}, \boldsymbol{s}') = (s_1' \equiv (s_1 \vee (s_2' \wedge s_3'))) \wedge (s_4' \equiv ((s_2 \wedge s_3) \vee s_4))$$

5. (d)

The initial state is

$$(s_1, s_2, s_3, s_4) = (0, 1, 0, 0)$$

Thus the initial-state characteristic function is

$$R_0 = \neg s_1 \wedge s_2 \wedge \neg s_3 \wedge \neg s_4$$

Using $T_{\exists}$, the reached state sets are:

$$R_1 = \neg s_4 \wedge (s_1 \equiv (s_2 \wedge s_3))$$

$$R_2 = s_1 \equiv ((s_2 \wedge s_3) \vee s_4)$$

$$R_3 = R_2 = s_1 \equiv ((s_2 \wedge s_3) \vee s_4)$$

Therefore, the fixed point is reached at iteration 2.

Since all reachable states satisfy

$$s_1 \equiv ((s_2 \wedge s_3) \vee s_4)$$

The output functions are

$$z_1 = s_1$$

and

$$z_2 = (s_2 \wedge s_3) \vee s_4$$

Thus, for all reachable states,

$$z_1 \equiv z_2$$

Therefore, C_1 and C_2 are equivalent under the given initial state.

5. (e)

Now the initial state is

$$(s_1, s_2, s_3, s_4) = (0, 1, 1, 0)$$

The initial-state characteristic function is

$$R_0 = \neg s_1 \wedge s_2 \wedge s_3 \wedge \neg s_4$$

Using $T_{\exists}$, the reached state sets are:

$$\begin{aligned} R_1 &= R_0 \vee (s_4 \wedge (s_1 \equiv (s_2 \wedge s_3))) \\ &= (\neg s_1 \wedge s_2 \wedge s_3 \wedge \neg s_4) \vee (s_4 \wedge (s_1 \equiv (s_2 \wedge s_3))) \\ R_2 &= (\neg s_1 \wedge s_2 \wedge s_3 \wedge \neg s_4) \vee (s_4 \wedge (s_1 \equiv (s_1 \vee (s_2 \wedge s_3)))) \\ R_3 &= R_2 \end{aligned}$$

Therefore, the fixed point is reached at iteration 2.

However, the initial state itself is reachable. At

$$(s_1, s_2, s_3, s_4) = (0, 1, 1, 0)$$

we have

$$z_1 = s_1 = 0$$

but

$$z_2 = (s_2 \wedge s_3) \vee s_4 = 1$$

Thus,

$$z_1 \neq z_2$$

Therefore, C_1 and C_2 are not equivalent under the new initial state.